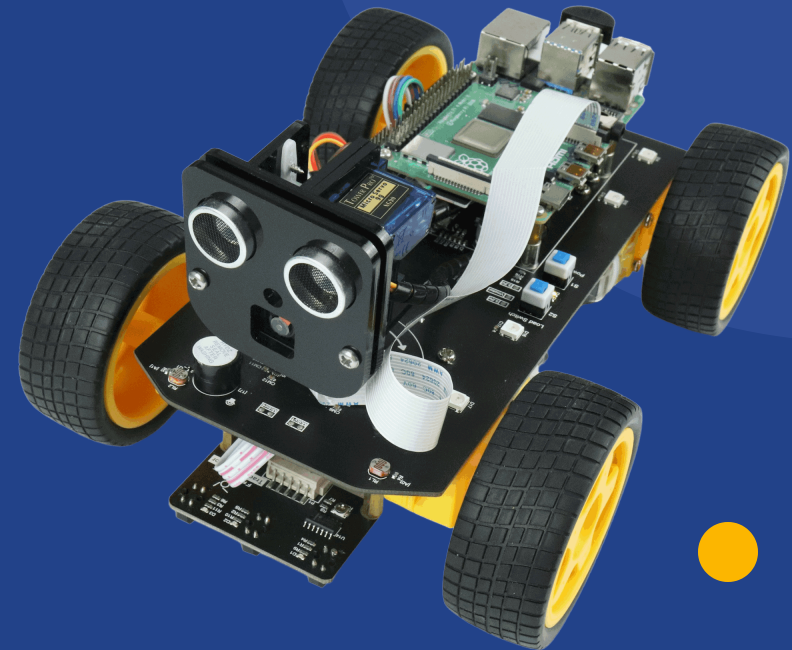




LESSON 11 – CAMERA GUI • BONUS

# Machine Learning with Raspberry Pi & Smart Car

One small window: **live camera**, **head buttons**, and a **face-tracking** checkbox — built from one PyQt5 file.



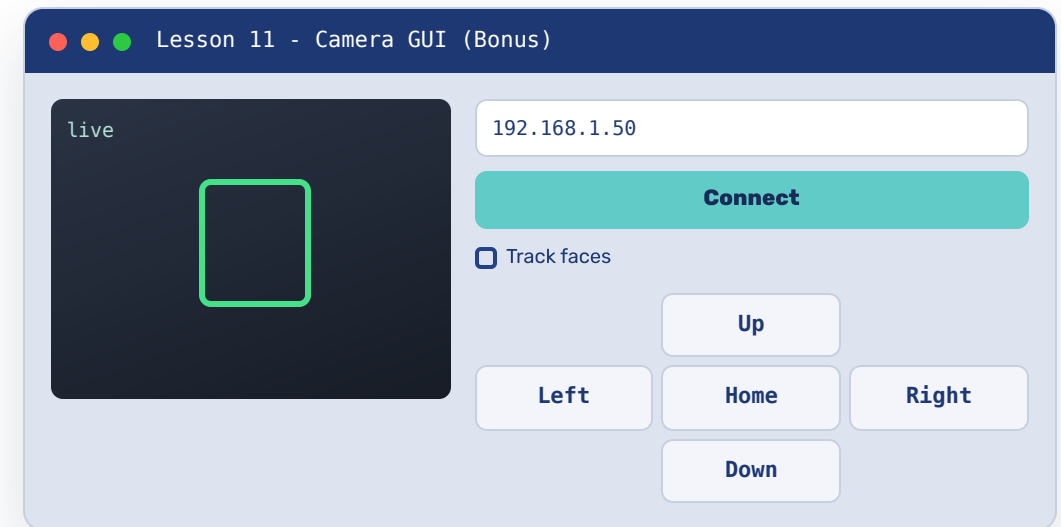


## LESSON 11 – CAMERA GUI

# Put it all in one window

No more separate terminal scripts – the camera and the head controls live in one app you can click.

- Until now, video and tracking ran as **separate scripts** in the terminal.
- Today: a single **desktop window** – live preview, head buttons, a checkbox.
- Same plumbing as before – **sockets, JPEG frames, CMD\_SERVO** – now behind a GUI.





## LESSON 11 – CAMERA GUI

# What you'll be able to do

Four verbs: **run** it, **control** the head, **understand** the video path, and **locate** each step in the code.



### Run

Launch the GUI and see the live preview.



### Control

Pan & tilt the head with five buttons.



### Understand

Capture → stream → decode → display.



### Locate

Point at each step inside one file.

One file does it all: `lesson_11_camera_gui.py` – one window class plus a handful of plain functions.



## LESSON 11 – CAMERA GUI • THE MENTAL MODEL

# How a frame reaches your screen

The Pi captures and streams; your computer decodes and shows. Nothing is saved to disk.



← **Commands go back on port 5000**: servo moves travel from the window to the car. **Two connections, two ports** – video in, commands out.



## LESSON 11 – CAMERA GUI • THE MENTAL MODEL

# One class, plain functions, one thread

A PyQt5 window needs a class – everything else stays the familiar lesson style.

- **One class** – `CameraWindow` – builds the window.
- Everything else is a **plain function**: connect, read, send, track.
- A **background thread** reads video so the buttons stay clickable.



### WHY A THREAD?

Reading pictures from the network is **slow**. Doing it in the background keeps the buttons **clickable** while video keeps flowing.



## BUILD THE FILE • CHUNK 1 / 15

# Start the file: imports

`car_setup` first — so `from command import COMMAND` just works.

- `socket` + `struct` read the stream.
- `threading` runs the background reader.
- `cv2` / `numpy` handle pictures; **PyQt5** builds the window.

```
lesson_11_camera_gui.py

"""Lesson 11: Camera GUI (Bonus)."""

import car_setup          # adds the car's code folders to the path

import socket             # talk to the Pi over the network
import struct             # read the 4-byte picture size
import sys
import threading          # receive video in the background

import cv2                # OpenCV: decode pictures, find faces
import numpy as np        # turn raw bytes into an image
from PyQt5 import QtCore, QtGui, QtWidgets  # the window

from Command import COMMAND # command names the server knows
```



## BUILD THE FILE • CHUNK 2 / 15

# Settings we can tune

Ports, frame size, pan vs tilt, and the **safe angle ranges**.

- Pan is channel `"0"`, tilt is channel `"1"`.
- Angles clamped: **pan 0–180, tilt 80–180**.
- `DEAD_ZONE` stops the head from **jittering**.

```
lesson_11_camera_gui.py

# ---- settings ----
VIDEO_PORT = 8000          # receive pictures here
COMMAND_PORT = 5000        # send servo commands here
FRAME_WIDTH = 400
FRAME_HEIGHT = 300
PAN_SERVO = "0"            # left/right head servo
TILT_SERVO = "1"           # up/down head servo
PAN_MIN, PAN_MAX = 0, 180  # degrees
TILT_MIN, TILT_MAX = 80, 180
SERVO_STEP = 4             # degrees moved per step
DEAD_ZONE = 0.15           # ignore tiny offsets
CASCADE_PATH = car_setup.CODE_DIR / "Client" / "haarcascade_frontalface_default.xml"
```



## BUILD THE FILE • CHUNK 3 / 15

# Shared state

Global because the **thread** and the **window** both touch them.

- The two **sockets** and the **latest frame**.
- A `Lock` = a "talking stick" for the frame.
- Head **angles** + two on/off **flags**.

```
lesson_11_camera_gui.py

# shared state used by the background video thread
cmd = COMMAND()          # names, like cmd.CMD_SERVO
face_cascade = cv2.CascadeClassifier(str(CASCADE_PATH))

video_socket = None       # receives pictures
video_file = None         # that connection, read like a file
command_socket = None     # sends servo commands
latest_frame = None       # the most recent picture
frame_lock = threading.Lock() # keeps the frame safe

running = False           # True while the thread should run
track_faces = False       # True when the box is ticked
pan_angle = 90.0          # current left/right angle
tilt_angle = 90.0         # current up/down angle
```



## BUILD THE FILE • CHUNK 4 / 15

# connect(): open two sockets

Pictures in on **8000**, commands out on **5000** – the car server's two ports.

- **Connection 1** – video in on port 8000.
- **Connection 2** – commands out on port 5000.
- `makefile("rb")` reads an exact number of bytes.

```
lesson_11_camera_gui.py

def connect(pi_ip):
    """Open the video and command sockets."""
    global video_socket, video_file, command_socket
    # Connection 1: pictures come in on port 8000.
    video_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    video_socket.connect((pi_ip, VIDEO_PORT))
    video_file = video_socket.makefile("rb") # read like a file

    # Connection 2: servo commands go out on port 5000.
    command_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    command_socket.connect((pi_ip, COMMAND_PORT))
```



## BUILD THE FILE • CHUNK 5 / 15

# read\_frame(): size first, then the picture

A 4-byte size, then that many JPEG bytes – exactly like Lesson 6.

- Read **4 bytes** → the picture's size.
- A short or **zero** read → stream ended.
- `imdecode` → image, **no file on disk**.

```
lesson_11_camera_gui.py

def read_frame():
    """Read one picture: a 4-byte size, then JPEG bytes."""
    header = video_file.read(4)          # the picture's size
    if len(header) != 4:
        return None
    length = struct.unpack("<L", header)[0]  # bytes -> number
    if length == 0:
        return None
    jpeg_data = video_file.read(length)    # exactly that many
    if len(jpeg_data) != length:
        return None
    return cv2.imdecode(np.frombuffer(jpeg_data, dtype=np.uint8),
                        cv2.IMREAD_COLOR)
```



## BUILD THE FILE • CHUNK 6 / 15

# send\_servo() & clamp()

A head command is just text: `CMD_SERVO#0#95\n`.

- Name `#` channel `#` angle, ending in a **newline**.
- Not connected yet? **Do nothing**.
- `clamp()` keeps every angle in its **safe range**.

```
lesson_11_camera_gui.py

def send_servo(channel, angle):
    """Send one servo command, e.g. CMD_SERVO#0#95."""
    if command_socket is None:
        return # not connected yet
    message = cmd.CMD_SERVO + "#" + channel + "#" \
        + str(int(angle)) + "\n"
    command_socket.send(message.encode("utf-8"))

def clamp(value, low, high):
    """Keep a number between low and high."""
    return max(low, min(high, value))
```



## BUILD THE FILE • CHUNK 7 / 15

# track\_face(): nudge toward center

Offset is **-1** at the edges and **0** in the middle – move a small step the right way.

- How far is the face from the **middle**?
- Inside the **dead zone** → don't move.
- Else nudge pan & tilt by `SERVO_STEP`.

```
lesson_11_camera_gui.py

def track_face(face):
    """Turn the head so the face moves toward the middle."""
    global pan_angle, tilt_angle
    x, y, w, h = face
    # -1 at left/top, +1 at right/bottom, 0 in the middle.
    offset_x = ((x + w / 2) / FRAME_WIDTH - 0.5) * 2
    offset_y = ((y + h / 2) / FRAME_HEIGHT - 0.5) * 2
    if abs(offset_x) < DEAD_ZONE and abs(offset_y) < DEAD_ZONE:
        return # already centred
    pan_angle = clamp(pan_angle + SERVO_STEP * offset_x, PAN_MIN, PAN_MAX)
    tilt_angle = clamp(tilt_angle - SERVO_STEP * offset_y, TILT_MIN, TILT_MAX)
    send_servo(PAN_SERVO, pan_angle)
    send_servo(TILT_SERVO, tilt_angle)
```



## BUILD THE FILE • CHUNK 8 / 15

## video\_thread(): the background loop

The slow work runs here – that's **why** the buttons keep responding.

- Read a frame; if tracking, **box every face**.
- Follow the **biggest** with `max(...)`.
- Store it under the **lock**, safely.

```
lesson_11_camera_gui.py

def video_thread():
    """Read a picture, optionally find faces, store it."""
    global latest_frame
    while running:
        frame = read_frame()
        if frame is None:
            break
        if track_faces:
            # box faces, follow biggest
            gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
            faces = face_cascade.detectMultiScale(gray, 1.3, 5)
            for (x, y, w, h) in faces:
                cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
            if len(faces) > 0:
                track_face(max(faces, key=lambda f: f[2] * f[3]))
        with frame_lock:
            # hand it over safely
            latest_frame = frame
```



BUILD THE FILE • CHUNK 9 / 15 • THE WINDOW 1 OF 3

# CameraWindow: video + connect

`.clicked.connect(...)` means "when this is used, call that function."

- A **video label** the picture draws onto.
- An **IP box** + **Connect** button.
- A **Track faces** checkbox.

```
lesson_11_camera_gui.py

class CameraWindow(QtWidgets.QMainWindow):
    def __init__(self):
        super().__init__()          # PyQt sets up first
        self.setWindowTitle("Lesson 11 - Camera GUI (Bonus)")

        # The label the video picture is drawn onto.
        self.video_label = QtWidgets.QLabel("Not connected")
        self.video_label.setFixedSize(FRAME_WIDTH, FRAME_HEIGHT)
        self.video_label.setAlignment(QtCore.Qt.AlignCenter)

        # An IP box and a Connect button.
        self.ip_box = QtWidgets.QLineEdit("192.168.1.50")
        connect_button = QtWidgets.QPushButton("Connect")
        connect_button.clicked.connect(self.on_connect)

        # A tick box to turn face tracking on and off.
        self.track_checkbox = QtWidgets.QCheckBox("Track faces")
        self.track_checkbox.stateChanged.connect(self.on_track_changed)
```

BUILD THE FILE • CHUNK 10 / 15 • THE WINDOW 2 OF 3

# The five head buttons

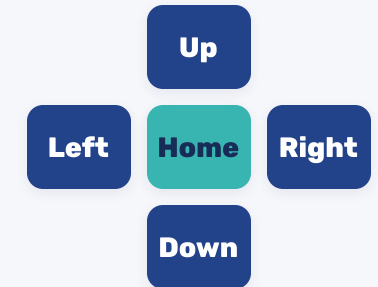
Each button calls `nudge()` with which servo and how far, laid out as a plus-pad.

```
lesson_11_camera_gui.py

# Five buttons to move the head by hand.
up = QtWidgets.QPushButton("Up")
down = QtWidgets.QPushButton("Down")
left = QtWidgets.QPushButton("Left")
right = QtWidgets.QPushButton("Right")
home = QtWidgets.QPushButton("Home")

# lambda passes the servo + step to nudge().
up.clicked.connect(lambda: self.nudge(TILT_SERVO, +SERVO_STEP))
down.clicked.connect(lambda: self.nudge(TILT_SERVO, -SERVO_STEP))
left.clicked.connect(lambda: self.nudge(PAN_SERVO, -SERVO_STEP))
right.clicked.connect(lambda: self.nudge(PAN_SERVO, +SERVO_STEP))
home.clicked.connect(self.go_home)

pad = QtWidgets.QGridLayout() # plus-sign grid
pad.addWidget(up, 0, 1)
pad.addWidget(left, 1, 0)
pad.addWidget(home, 1, 1)
pad.addWidget(right, 1, 2)
pad.addWidget(down, 2, 1)
```



`addWidget(widget, row, col)`



BUILD THE FILE • CHUNK 11 / 15 • THE WINDOW 3 OF 3

# Lay it out & start the timer

A timer redraws the newest picture every **30 ms** (~33 fps).

- Controls down the **right** ( `QVBoxLayout` ).
- Video on the **left** ( `QHBoxLayout` ).
- A `QTimer` keeps the picture **fresh**.

```
lesson_11_camera_gui.py

# Stack the controls down the right-hand side.
controls = QtWidgets.QVBoxLayout()
controls.addWidget(self.ip_box)
controls.addWidget(connect_button)
controls.addWidget(self.track_checkbox)
controls.addLayout(pad)
controls.addStretch(1)          # push them to the top

layout = QtWidgets.QHBoxLayout() # video left, controls right
layout.addWidget(self.video_label)
layout.addLayout(controls)
container = QtWidgets.QWidget()
container.setLayout(layout)
self.setCentralWidget(container)

# Redraw the newest picture about every 30 ms.
self.timer = QtCore.QTimer()
self.timer.timeout.connect(self.refresh_video)
self.timer.start(30)
```



## BUILD THE FILE • CHUNK 12 / 15

## on\_connect(): connect, then start the thread

A bad IP shows a message instead of crashing – then the background reader starts.

- `connect()` with the IP from the box.
- Bad IP → `OSError` → show a message.
- Start a **daemon thread** (stops with the app).

```
lesson_11_camera_gui.py

def on_connect(self):
    """Connect, then start the background video thread."""
    global running
    try:
        connect(self.ip_box.text()) # IP from the box
    except OSError as error:
        self.video_label.setText("Could not connect:\n" + str(error))
        return
    running = True
    thread = threading.Thread(target=video_thread)
    thread.daemon = True # stop when the app closes
    thread.start()
```



## BUILD THE FILE • CHUNK 13 / 15

# Button handlers: toggle, nudge, home

Both the buttons and `track_face()` reuse `send_servo()` and `clamp()`.

- `on_track_changed` → flip the flag.
- `nudge` → one servo, one step, clamped.
- `go_home` → recentre to **90°**.

```
lesson_11_camera_gui.py

def on_track_changed(self):
    global track_faces
    track_faces = self.track_checkbox.isChecked()

def nudge(self, channel, change):
    global pan_angle, tilt_angle
    if channel == PAN_SERVO:
        pan_angle = clamp(pan_angle + change, PAN_MIN, PAN_MAX)
        send_servo(PAN_SERVO, pan_angle)
    else:
        tilt_angle = clamp(tilt_angle + change, TILT_MIN, TILT_MAX)
        send_servo(TILT_SERVO, tilt_angle)

def go_home(self):
    global pan_angle, tilt_angle
    pan_angle, tilt_angle = 90.0, 90.0
    send_servo(PAN_SERVO, pan_angle)
    send_servo(TILT_SERVO, tilt_angle)
```



## BUILD THE FILE • CHUNK 14 / 15

# refresh\_video(): copy, convert, show

The colour swap (**BGR** → **RGB**) is why faces look natural, not blue-tinted.

- Quick **copy under the lock**.
- No picture yet → **return**.
- Wrap pixels in a `QImage`, draw it.

```
lesson_11_camera_gui.py

def refresh_video(self):
    """Copy the newest picture and draw it in the window."""
    with frame_lock:
        frame = None if latest_frame is None else latest_frame.copy()
    if frame is None:
        return # no picture yet
    # OpenCV is Blue-Green-Red; Qt expects Red-Green-Blue.
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    height, width, _ = rgb.shape
    image = QtGui.QImage(rgb.data, width, height, 3 * width,
                        QtGui.QImage.Format_RGB888)
    self.video_label.setPixmap(QtGui.QPixmap.fromImage(image))
```



## BUILD THE FILE • CHUNK 15 / 15

# destroy() & the main block

PyQt runs its own loop – so it's `try / finally`, but the clean-up habit is the same.

- Stop the thread, close the **sockets**.
- One `QApplication`, show the window.
- `finally` always **cleans up**.

```
lesson_11_camera_gui.py

def destroy():
    """Stop the thread and close the connections."""
    global running
    running = False          # tell the thread to stop
    if video_file is not None:
        video_file.close()
    if video_socket is not None:
        video_socket.close()
    if command_socket is not None:
        command_socket.close()

if __name__ == "__main__":
    app = QtWidgets.QApplication(sys.argv)  # every PyQt app needs one
    window = CameraWindow()
    window.show()
    try:
        app.exec_()                # runs until you close it
    finally:
        destroy()                  # always clean up
```



## LESSON 11 – CAMERA GUI

# We built the whole file

One class for the window, plain functions for the work, one thread for the video.



## lesson\_11\_camera\_gui.py

imports → settings → shared state → connect() → read\_frame() → send\_servo() · clamp() → track\_face() → video\_thread() →  
CameraWindow → handlers → refresh\_video() → destroy() → main



A graphical window needs **one class** — `CameraWindow`. Everything else is the same **plain-function** style you've used all course.

## LESSON 11 – CAMERA GUI

# Run it

Start the server on the Pi, run the GUI on your computer, then connect from the window.

### Step 1 – on the Pi, start the car's server

```
● ● ● Pi terminal  
  
cd Smart-Car-Level-3/Code/Server  
sudo python3 main.py
```

### Step 2 – on your computer, run the GUI

```
● ● ● terminal  
  
cd Smart-Car-Level-3/Code/User  
python lesson_11_camera_gui.py
```

1 Type your Pi's IP in the box.

2 Click **Connect** – the preview starts.

3 Tick **Track faces** to follow a face.

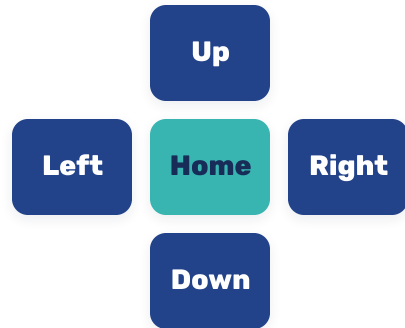
Box defaults to `192.168.1.50` – change it. Ports **5000** & **8000** must be reachable.



## LESSON 11 – CAMERA GUI • CONTROL

# Moving the camera head

Each button nudges one servo by `SERVO_STEP` (4°), sent as a `CMD_SERVO` command.



`CMD_SERVO#0#<pan> · #1#<tilt>`

`Up · Down` **Tilt**, channel `"1"` – up **+4°**, down **-4°**.

`Left · Right` **Pan**, channel `"0"` – left **-4°**, right **+4°**.

`Home` Recentre pan and tilt to **90°**.



## LESSON 11 – CAMERA GUI

# Try it yourself

`SERVO_STEP` and `DEAD_ZONE` are the fun dials – feel twitchy vs. smooth.

- Change `SERVO_STEP` – bigger = **faster, jerkier**; smaller = smoother.
- Change `DEAD_ZONE` – bigger ignores more wobble.
- Resize the frame with `FRAME_WIDTH` / `FRAME_HEIGHT` .

### CHALLENGE

#### Add a snapshot button

Save the current frame as a **timestamped .png** – or add a small label showing the **live pan/tilt angles**.



## LESSON 11 – CAMERA GUI

# Common issues

Most issues are the **wrong IP** or the **server not started** – check those first.



### "Could not connect"

Wrong IP, or ports 5000/8000 blocked. Fix it and click Connect again.



### Image is black or frozen

Check the camera cable; restart the GUI and the Pi server.



### Face tracking does nothing

Tick **Track faces**; the cascade must load from `Code/Client/`.



### Detection feels slow

It runs on **your computer**. Improve lighting and face the camera.



## Check two things

Is the **server running** on the Pi, and is the **IP right**? That fixes most connection problems.



## LESSON 11 – CAMERA GUI

# You built the course capstone!

One window ties together capture, streaming, decoding, display, and control.

- ✓ Wrapped video + control in **one PyQt5 window**
- ✓ Used a **background thread** to stay responsive
- ✓ Controlled the head **by hand** and **by face**



### MAKE IT YOUR OWN

## A base for your final project

The GUI follows **where** a face is, not **who** it is – a clean starting point to build on.



## • COACH ONLY • HIDDEN SLIDE

# End-of-class checklist

Don't present this slide. The build is long – pace the room and consider a break after the thread.

## 1 Demo the responsiveness point

With tracking on, the buttons still click instantly – that's the **thread** earning its keep. Make the idea concrete.

## 2 Post the after-class recap

Summarise on the class page; log absent students in the **#attendance** Discord channel.

## 3 Point toward final projects

This file is a strong base – nudge students toward the snapshot / angle-label extensions.

### EVERY CLASS

✓ Post a class announcement

✓ Introductions if needed

✓ Note absences in **#attendance**

### BONUS LESSON

Lesson 11 is a **bonus capstone** with no fixed slot in the course – run it **in place of** another lesson, **in addition to** the regular set, or as a **standalone finale**. Slot it in wherever it fits your class.